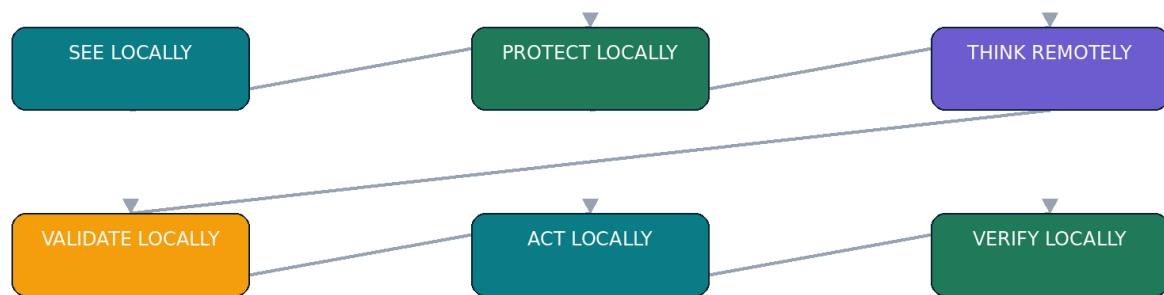


N-EYE

SOFTWARE ARCHITECTURE & ENGINEERING MASTERBOOK

HLD + LLD + Security + Privacy + Testing + Delivery + AI Engineering Control Plane

N-Eye Canonical Control Loop



Architecture baseline v1.0 | 28 August 2026 | SIH prototype first, company-scale seams preserved

CORE PROMISE Give AI enough context to be useful without automatically giving it the user's private environment or unrestricted action authority.

How to Read This Masterbook

Every major concept is presented in two lanes: precise engineering meaning and plain-language meaning.

TECHNICAL

The technical lane defines runtime placement, contracts, ownership, failure behavior, testability, and non-negotiable invariants.

HUMAN EXPLANATION

The human lane explains what the same idea means, why it exists, and what could go wrong if we ignore it.

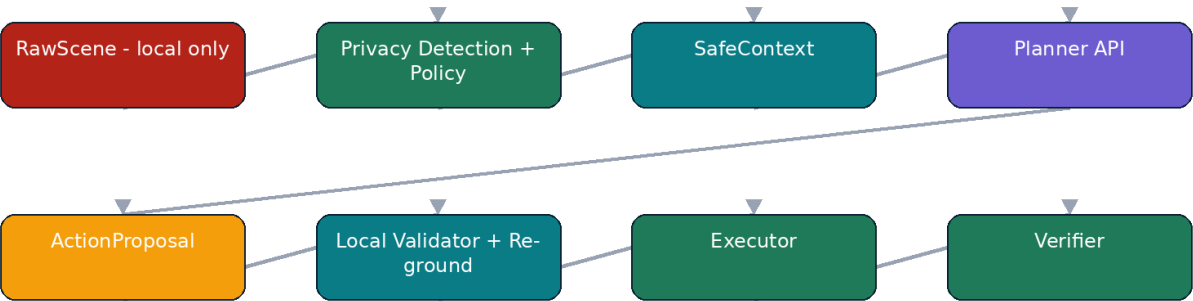
EVIDENCE DISCIPLINE Statements are separated into SOURCE FACT, VERIFIED EXTERNAL FACT, ARCHITECTURE DECISION, and EXPERIMENT REQUIRED. Targets are never presented as measured N-Eye results.

- Primary internal authority: N-Eye SIH 2026 Human-First Engineering Masterbook (95 pages).
- Continuity authority: N-Eye_context.md.
- Supporting internal sources: execution masterbook, prototype blueprints, company blueprint, N-EyeDeep, and the 158-page build guide.
- Changing external facts must be rechecked against authoritative documentation before implementation freeze.

Executive Architecture

The entire system in one page.

Data and Control Flow



TECHNICAL

N-Eye is a Chrome-first edge-cloud agent. The local side observes the page, escalates perception only when needed, protects private data, validates remote proposals, resolves private tokens, executes locally, and verifies state change.

HUMAN EXPLANATION

The browser does the seeing, hiding, checking, clicking, and confirming. The cloud is asked only for the thinking step, and it never receives automatic control of the browser.

ULTIMATE INVARIANT Raw page/private values stay local by default; SafeContext is the normal outbound contract; ActionProposal is untrusted advice; important actions succeed only after observed verification.

1. Product Definition & Scope

What exactly is being built, what is deliberately excluded, and what “done” means.

System identity

TECHNICAL

A privacy-preserving browser agent and trust layer that minimizes disclosure before remote reasoning and retains final browser authority locally.

HUMAN EXPLANATION

Think of N-Eye as a security guard between your private browser and AI. It shares only what is needed and checks every suggested action before allowing it.

ARCHITECTURE DECISION The SIH prototype is a complete vertical slice, not a collection of AI demos.

VERIFICATION / EXIT GATE A real task completes end-to-end with inspectable privacy evidence and verified page change.

Prototype vs company

TECHNICAL

Prototype scope is Chrome MV3 + thin planner API + controlled test portal + benchmark/evidence harness. Company scope later adds hardened identity, policy, audit, multi-browser and deployment controls.

HUMAN EXPLANATION

We prove the core engine first. We do not spend the 20-day build creating enterprise infrastructure that does not prove the core idea.

ARCHITECTURE DECISION Preserve interfaces for evolution; do not implement company-grade infrastructure prematurely.

VERIFICATION / EXIT GATE Every proposed feature must improve the vertical slice, privacy proof, reliability, or measured performance.

Non-goals

TECHNICAL

No foundation-model training, no always-on giant VLM, no arbitrary remote JavaScript/selectors, no raw screenshot default, no broad browser parity, no Kubernetes, no production compliance claims.

HUMAN EXPLANATION

We refuse impressive-looking work that adds risk but does not help us prove N-Eye.

ARCHITECTURE DECISION YAGNI is a safety principle here, not just a productivity principle.

VERIFICATION / EXIT GATE Architecture review rejects non-critical complexity.

2. Engineering Principles

Rules that prevent architecture drift.

Correctness before cleverness

TECHNICAL

Prefer explicit typed contracts, deterministic validators, bounded responsibilities, and measurable exit gates over hidden magic.

HUMAN EXPLANATION

If we cannot explain what a component receives, produces, and proves, it is not ready to build.

ARCHITECTURE DECISION KISS + least privilege + fail-safe defaults outrank novelty.

VERIFICATION / EXIT GATE Every component has owner, input, output, failure mode, and test.

SOLID without class inflation

TECHNICAL

Apply SRP, interface segregation and dependency inversion at module boundaries; prefer composition; use classes only when lifecycle/state/encapsulation justify them.

HUMAN EXPLANATION

We use clean boundaries, not hundreds of tiny classes just to look “enterprise.”

ARCHITECTURE DECISION Functional modules are acceptable for stateless transformations; stateful vault/registry/orchestrator may use explicit objects.

VERIFICATION / EXIT GATE Static dependency graph has no cycles and package responsibilities remain cohesive.

Architecture change control

TECHNICAL

Agents may recommend improvements but cannot silently change trust boundaries, schemas, dependencies, persistence, or remote egress.

HUMAN EXPLANATION

AI coding tools can suggest a better road, but they cannot secretly move the destination.

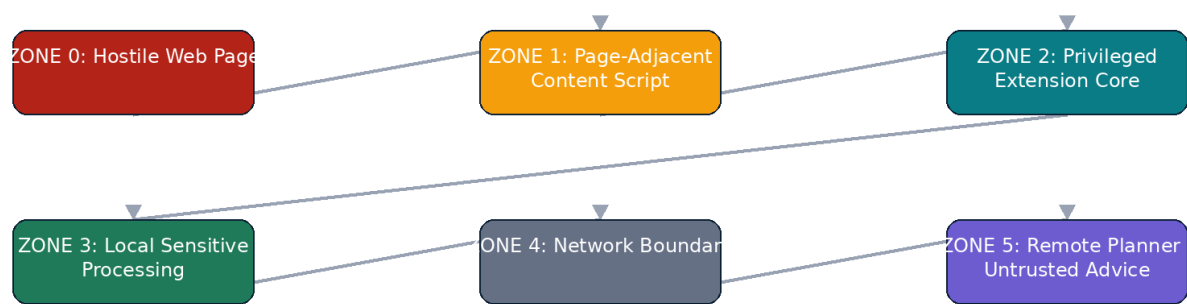
ARCHITECTURE DECISION Material changes require recommendation -> human approval -> ADR -> updated architecture -> bounded implementation task.

VERIFICATION / EXIT GATE No architecture-impacting diff merges without an ADR or explicit human approval.

3. Trust Model

Authority is more granular than “local trusted, cloud untrusted.”

Trust Zones and Authority Boundaries



Zone model

TECHNICAL

Zone 0 webpage hostile; Zone 1 content script page-adjacent; Zone 2 privileged extension core; Zone 3 local sensitive processing; Zone 4 network boundary; Zone 5 remote planner untrusted.

HUMAN EXPLANATION

Even code running on the computer is not equally trusted. The script touching the webpage is closer to attacker-controlled content than the private vault.

ARCHITECTURE DECISION Trust decreases near webpage and network; authority increases only in tightly controlled local modules.

VERIFICATION / EXIT GATE Threat tests attempt forged messages, malicious DOM, cross-origin replay, and planner manipulation.

Page is data, not instruction

TECHNICAL

DOM text, OCR text, alt text, PDF text and canvas content are untrusted observations. They never override system policy.

HUMAN EXPLANATION

A website can literally write “ignore your rules.” N-Eye treats that sentence as webpage content, not as an order.

ARCHITECTURE DECISION Prompt-injection defenses are architectural: constrained context, constrained output,

least privilege, local authorization.

VERIFICATION / EXIT GATE Injected page text cannot produce an action outside ActionProposal schema or local policy.

Local authority

TECHNICAL

Only locally validated actions reach the executor; token resolution happens after target/origin/risk checks.

HUMAN EXPLANATION

The cloud may say “click e17,” but the browser checks that e17 is still the right button before touching it.

ARCHITECTURE DECISION No direct planner-to-executor path exists.

VERIFICATION / EXIT GATE Tests prove malformed, stale, risky, cross-origin and unknown-target proposals are blocked.

4. High-Level Runtime Architecture

Where code runs in Chrome and why.

Content script

TECHNICAL

Collects deliberately limited visible/interactable structure, maintains live element references, performs approved DOM actions, and emits observations.

HUMAN EXPLANATION

This is N-Eye’s eyes and hands inside the webpage, but it is not allowed to hold the most sensitive authority.

ARCHITECTURE DECISION Never place planner keys, vault values, or unrestricted privileged APIs here.

VERIFICATION / EXIT GATE Malicious page cannot extract server credentials or bypass local validator through content-script messaging.

Service worker

TECHNICAL

Coordinates task state, messaging, permissions, planner calls and extension lifecycle; designed for suspension/restart. It has no DOM access.

HUMAN EXPLANATION

This is the coordinator. Chrome may put it to sleep, so important state cannot depend on a forever-running background process.

ARCHITECTURE DECISION Orchestration must be resumable/idempotent where needed.

VERIFICATION / EXIT GATE Suspend/restart tests do not corrupt task state or duplicate high-risk actions.

Side panel

TECHNICAL

Trusted extension UI for task input, state, privacy summary, safe-context inspector, confirmations and evidence mode.

HUMAN EXPLANATION

This is the control room the user and judges can understand. It shows what N-Eye is doing without exposing secrets in debug output.

ARCHITECTURE DECISION UI observes state; it does not become the source of security truth.

VERIFICATION / EXIT GATE Closing/reopening panel does not bypass task/policy state.

Worker/offscreen

TECHNICAL

Runs OCR/image preprocessing and narrow hidden DOM/window work only where justified; expensive work stays off main UI path.

HUMAN EXPLANATION

This is the local workshop for heavier image work so the interface does not freeze.

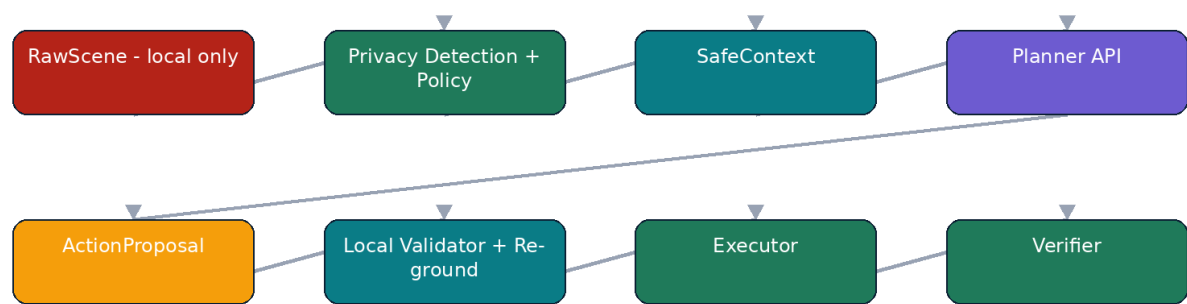
ARCHITECTURE DECISION Use worker first for compute; offscreen only for APIs requiring DOM/window.

VERIFICATION / EXIT GATE OCR workload does not block task UI and has a deterministic fallback.

5. Canonical Data Flow

The protected path from page to action.

Data and Control Flow



RawScene

TECHNICAL

Local-only aggregate of visible structure, geometry, OCR/visual findings and privacy candidates. It is intentionally non-serializable to planner transport.

HUMAN EXPLANATION

This is everything N-Eye temporarily knows before it has cleaned the information. It is not allowed to leave the private side.

ARCHITECTURE DECISION Network client has no API accepting RawScene or arbitrary object.

VERIFICATION / EXIT GATE Compile/runtime tests and canary byte scans prove RawScene cannot be sent.

SafeContext

TECHNICAL

Strict allowlist object containing sanitized goal, safe page metadata, safe elements/text, token roles and explicitly approved safe crop references.

HUMAN EXPLANATION

This is the small safe note we hand to the cloud after removing unnecessary private details.

ARCHITECTURE DECISION SafeContext is a security boundary, not a UI label.

VERIFICATION / EXIT GATE Actual serialized request is inspected; forbidden canaries are absent.

ActionProposal

TECHNICAL

Constrained remote response such as CLICK, TYPE_TOKEN, SCROLL, SELECT, WAIT, ASK_USER, COMPLETE.

HUMAN EXPLANATION

The AI can choose from a small menu of suggestions; it cannot send browser code.

ARCHITECTURE DECISION Malformed/unknown actions fail schema validation.

VERIFICATION / EXIT GATE Fuzzed planner outputs cannot reach executor.

6. Observation & Element Identity

How N-Eye sees a changing webpage without scraping everything.

Visible-element collector

TECHNICAL

Collect role/tag, sanitized label candidate, input semantics, enabled/selected state, geometry, viewport metadata and page epoch; avoid generic values, hidden nodes, storage, cookies, full HTML and data-* dumps.

HUMAN EXPLANATION

We read only the parts needed to understand and act, not the entire website.

ARCHITECTURE DECISION Observer is task-oriented, not a scraper.

VERIFICATION / EXIT GATE Synthetic hidden-leak page proves hidden secrets do not enter default observation.

Opaque element IDs

TECHNICAL

Each useful target receives session-local ID e.g. e17 mapped to a local live reference. Remote planner never owns CSS selectors.

HUMAN EXPLANATION

The cloud says “use button e17,” not “run this selector/code.”

ARCHITECTURE DECISION Element IDs expire with session/page identity and are never global identifiers.

VERIFICATION / EXIT GATE Unknown/expired IDs are rejected.

Target fingerprint

TECHNICAL

Fingerprint combines stable-enough semantics, geometry/state signals and page epoch to detect stale/mismatched targets.

HUMAN EXPLANATION

Before clicking, N-Eye asks “is this still the same button I planned for?”

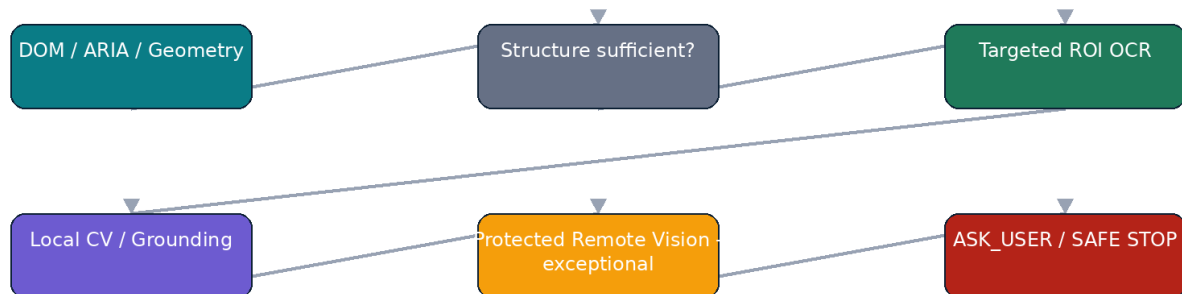
ARCHITECTURE DECISION Fingerprint is a guard, not an absolute identity guarantee; re-observation wins when uncertain.

VERIFICATION / EXIT GATE Dynamic SPA replaces target after planning and stale action is blocked.

7. Adaptive Perception

Use the cheapest trustworthy sensor first.

Adaptive Perception Escalation



Structure-first decision

TECHNICAL

DOM/ARIA/geometry answers the question when confidence and task requirements are sufficient.

HUMAN EXPLANATION

If the browser already knows something is a Continue button, taking a screenshot just wastes privacy and time.

ARCHITECTURE DECISION No screenshot is captured merely because vision exists.

VERIFICATION / EXIT GATE DOM-only scenario completes without screenshot capture.

Targeted OCR

TECHNICAL

When pixels contain missing text, capture smallest useful ROI, preprocess locally, OCR locally, preserve boxes/confidence/provenance, then pass OCR text through privacy pipeline.

HUMAN EXPLANATION

If useful text exists only in an image, we read only that small area on the device.

ARCHITECTURE DECISION OCR output is not automatically safe; it is another untrusted observation.

VERIFICATION / EXIT GATE Pixel-email canary is detected locally and absent from outbound payload.

Local visual grounding

TECHNICAL

For icon/canvas/non-text targets, use lightweight local CV/grounding only after benchmark proves need; WebGPU is acceleration, WASM/CPU correctness fallback.

HUMAN EXPLANATION

For controls the browser cannot describe, N-Eye uses local vision, but only when simpler methods fail.

ARCHITECTURE DECISION Model choice is EXPERIMENT REQUIRED; architecture must not depend on one model.

VERIFICATION / EXIT GATE At least one genuine visual-only task succeeds with measured resource/latency data.

8. Privacy Engine

Privacy is a pipeline, not one classifier.

Detection fusion

TECHNICAL

Combine browser semantics, deterministic patterns, OCR-derived text, optional local NER/visual detectors and task/site context. Preserve provenance and confidence.

HUMAN EXPLANATION

A password field, an email string, a face, and a company name are different privacy problems, so one detector cannot handle all of them well.

ARCHITECTURE DECISION Optional models enter only when benchmarked benefit exceeds cost.

VERIFICATION / EXIT GATE Per-class P/R/F1 reported on held-out synthetic dataset.

Privacy decisions

TECHNICAL

Policy outputs KEEP/ALLOW, MINIMIZE, GENERALIZE, TOKENIZE, MASK/REMOVE, NEVER_SEND, or ASK_USER according to class, task necessity and risk.

HUMAN EXPLANATION

N-Eye does not simply blur everything. It decides what the AI actually needs and protects the rest.

ARCHITECTURE DECISION Passwords, OTPs, API/session tokens and equivalent secrets are NEVER_SEND by default.

VERIFICATION / EXIT GATE Policy tests cover class x context x action matrix.

Sanitizer

TECHNICAL

Transforms allowed observations into protected forms and strips disallowed fields before SafeContext construction.

HUMAN EXPLANATION

Detection finds private information; sanitization is the step that actually removes or replaces it.

ARCHITECTURE DECISION Sanitizer is deterministic for security-critical transformations.

VERIFICATION / EXIT GATE Residual leakage tests search output bytes for planted canaries.

9. Private Token Vault

Preserve meaning without revealing value.

Token binding

TECHNICAL

Opaque token maps locally to value + privacy class + task + origin + allowed target semantics + creation/expiry + use constraints.

HUMAN EXPLANATION

The AI can know “[EMAIL_1] exists” without learning the email address itself.

ARCHITECTURE DECISION Bindings never serialize; default prototype vault is memory-local.

VERIFICATION / EXIT GATE Storage/network/log scan finds no raw vault values.

Resolution

TECHNICAL

TYPE_TOKEN resolves only after schema, task, origin, page epoch, target semantics, risk and expiry checks.

HUMAN EXPLANATION

The real email is inserted only at the final moment, after N-Eye proves it is going into the right field.

ARCHITECTURE DECISION Token resolution is authorization, not string replacement.

VERIFICATION / EXIT GATE Wrong-field, replay, stale and cross-origin token attacks fail.

Destruction

TECHNICAL

Task completion, cancellation, origin change, expiry or security fault destroys/invalidates bindings according to lifecycle policy.

HUMAN EXPLANATION

Private temporary memory should disappear when it is no longer needed.

ARCHITECTURE DECISION Prototype avoids persistent secret storage.

VERIFICATION / EXIT GATE Lifecycle tests prove token cannot be reused after invalidation.

10. SafeContext & Egress Guard

The most important network contract.

Strict schema

TECHNICAL

SafeContext uses explicit allowed fields; unknown fields rejected/stripped according to boundary policy; goal itself is sanitized.

HUMAN EXPLANATION

Even the user's task sentence can contain a secret, so the safe package must clean that too.

ARCHITECTURE DECISION Do not serialize arbitrary dictionaries/objects.

VERIFICATION / EXIT GATE Schema mutation/fuzz tests cannot smuggle extra secret-bearing fields.

Egress guard

TECHNICAL

Last local gate serializes only validated SafeContext, applies request-size limits, privacy-safe logging, canary scan in test builds, and blocks fail-closed on invariant violation.

HUMAN EXPLANATION

Right before anything leaves the device, one final checkpoint asks: "is this exactly the safe object we intended?"

ARCHITECTURE DECISION No bypass network helper may exist in sensitive packages.

VERIFICATION / EXIT GATE Repository search + tests prove planner transport has one controlled entry path.

Network metadata

TECHNICAL

Headers, URLs, query strings, error telemetry and logs are part of the privacy surface, not just JSON body.

HUMAN EXPLANATION

A secret can leak in a URL or error message even when the main payload is clean.

ARCHITECTURE DECISION Canary suite scans every egress/logging surface.

VERIFICATION / EXIT GATE CI fails on any forbidden canary outside approved local boundary.

11. Remote Planner

Reasoning service without browser ownership.

Thin API

TECHNICAL

FastAPI service validates SafeContext, authenticates request, applies size/time limits, calls provider adapter, validates structured output, returns ActionProposal, and sanitizes logs.

HUMAN EXPLANATION

The server is deliberately boring: receive safe context, ask a model, return one small structured suggestion.

ARCHITECTURE DECISION Provider API key remains server-side; server never receives vault implementation.

VERIFICATION / EXIT GATE Malformed/oversized/unauthenticated requests fail safely.

Provider abstraction

TECHNICAL

Planner interface isolates model/provider details from protocol and browser code.

HUMAN EXPLANATION

We can change AI providers later without rebuilding the browser security model.

ARCHITECTURE DECISION No provider-specific object leaks into ActionProposal contract.

VERIFICATION / EXIT GATE Mock and real providers pass same contract suite.

Failure policy

TECHNICAL

Timeout/network/malformed output -> bounded retry only where safe -> replan/ASK_USER/fail; never relax schema or privacy to recover.

HUMAN EXPLANATION

If the AI service fails, N-Eye becomes less powerful, not less safe.

ARCHITECTURE DECISION Recovery cannot convert SafeContext into raw screenshot fallback.

VERIFICATION / EXIT GATE Network-down demo uses mock planner with identical contracts.

12. Local Validation & Risk Policy

The browser decides whether a proposal may happen.

Validation pipeline

TECHNICAL

Check schema, action allowlist, task, origin, page epoch, target ID, fingerprint, visibility/interactability, token scope, action risk and confirmation requirement.

HUMAN EXPLANATION

A suggestion is not permission. N-Eye checks every important condition again locally.

ARCHITECTURE DECISION Validation returns ValidatedAction or explicit rejection reason; executor accepts only ValidatedAction.

VERIFICATION / EXIT GATE Type system and tests prevent raw ActionProposal execution.

Risk levels

TECHNICAL

LOW: reversible navigation/read-like; MEDIUM: form edits; HIGH: submit/delete/upload/financial/security changes; CRITICAL/unsupported: blocked or explicit user control. Exact matrix is policy-configured.

HUMAN EXPLANATION

Clicking a harmless tab is not the same as submitting a payment. N-Eye treats actions differently based on consequences.

ARCHITECTURE DECISION Risk taxonomy is an architecture decision and must be tuned against SIH scenarios.

VERIFICATION / EXIT GATE High-risk synthetic page requires confirmation and cannot auto-execute.

Re-grounding

TECHNICAL

Immediately before action, resolve current live target and compare expected semantics/state/epoch; stale mismatch causes rejection and re-observation.

HUMAN EXPLANATION

Webpages can change in milliseconds, so N-Eye checks the target one last time.

ARCHITECTURE DECISION No stale-coordinate execution.

VERIFICATION / EXIT GATE SPA race test proves replaced target is never clicked blindly.

13. Execution & Verification

Action executed is not task succeeded.

Executor

TECHNICAL

Content script performs one locally validated action using controlled implementations; no eval, arbitrary JS, unrestricted selector, shell or generic script action.

HUMAN EXPLANATION

N-Eye has a small set of safe hands, not a remote-control console.

ARCHITECTURE DECISION Executor API accepts only ValidatedAction.

VERIFICATION / EXIT GATE Static and runtime tests prove no free-form execution path.

Verification expectation

TECHNICAL

Before action, define expected state delta: URL, element appearance/disappearance, semantic state, value state, success message or task-specific predicate.

HUMAN EXPLANATION

Before clicking, we decide what success should look like.

ARCHITECTURE DECISION Verification criteria travel with local task state, not trusted from model alone.

VERIFICATION / EXIT GATE Tests include false-positive “click happened but page did not change.”

Verifier

TECHNICAL

Re-observe after action, compare expected delta, classify VERIFIED/FAILED/AMBIGUOUS; ambiguous -> re-observe/replan/ask.

HUMAN EXPLANATION

N-Eye checks the result with its own eyes instead of believing the AI saying “done.”

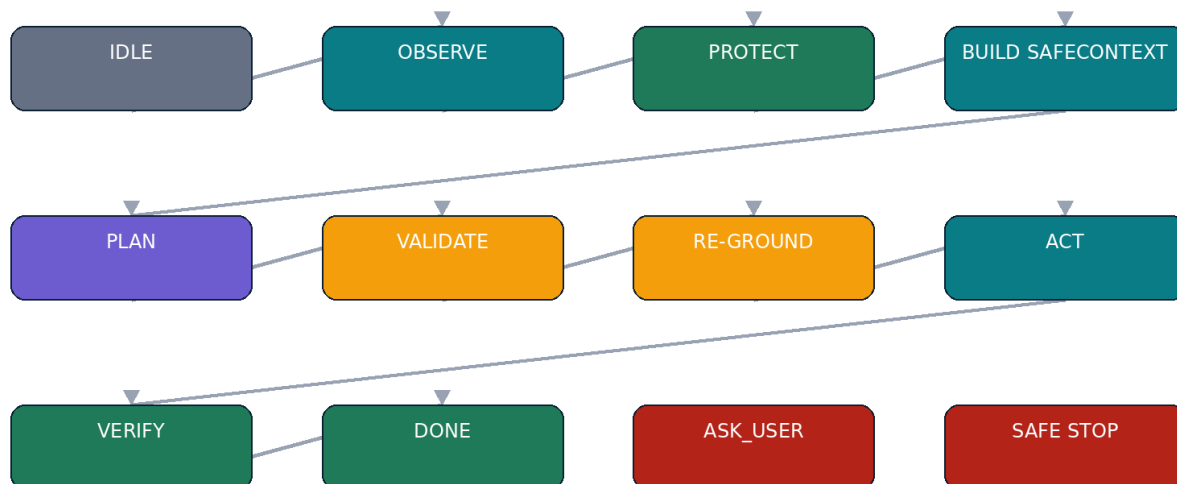
ARCHITECTURE DECISION No success-by-assumption.

VERIFICATION / EXIT GATE Verification failure never marks task complete.

14. State, Concurrency & Idempotency

Prevent races, duplicate actions and stale work.

Task State Machine



Task state machine

TECHNICAL

Explicit states
IDLE/OBSERVE/PERCEIVE/PROTECT/BUILD_SAFE_CON
TEXT/PLAN/VALIDATE/CONFIRM/REGROUND/ACT/
VERIFY/DONE plus blocked/cancel/failure branches.

HUMAN EXPLANATION

The system always knows which step it is in; it does not randomly call modules.

ARCHITECTURE DECISION Illegal transitions throw typed errors and are telemetry-visible.

VERIFICATION / EXIT GATE Transition table is unit tested.

Action idempotency

TECHNICAL

Each proposal/action attempt has unique ID;
coordinator prevents duplicate execution after
service-worker wake/retry.

HUMAN EXPLANATION

If Chrome pauses and resumes the extension, we must not accidentally click Submit twice.

ARCHITECTURE DECISION High-risk actions are at-most-once from orchestrator perspective unless user explicitly retries.

VERIFICATION / EXIT GATE Suspend/retry scenario produces one execution event.

Cancellation

TECHNICAL

User cancel propagates through planner/perception tasks where possible, invalidates pending actions/tokens and returns to safe state.

HUMAN EXPLANATION

The user can stop the agent, and old work cannot continue acting afterward.

ARCHITECTURE DECISION Cancellation is authority revocation.

VERIFICATION / EXIT GATE Cancel-during-plan and cancel-before-act tests block later execution.

15. Error Model & Safe Recovery

Failure is designed, not improvised.

Typed error taxonomy

TECHNICAL

Use categories: OBSERVATION, PERCEPTION, PRIVACY, PROTOCOL, NETWORK, PLANNER, VALIDATION, POLICY, EXECUTION, VERIFICATION, LIFECYCLE, INTERNAL. Include safe code, recoverability and redacted details.

HUMAN EXPLANATION

Instead of “something failed,” we know which layer failed and whether retry is safe.

ARCHITECTURE DECISION Errors never carry raw secrets into telemetry.

VERIFICATION / EXIT GATE Error serialization redaction tests use canaries.

Degradation ladder

TECHNICAL

Structure -> targeted OCR -> local CV -> protected exceptional fallback -> re-observe -> re-plan -> ASK_USER -> SAFE STOP. Authority never increases as confidence falls.

HUMAN EXPLANATION

When N-Eye becomes unsure, it asks for help or stops; it never becomes more reckless.

ARCHITECTURE DECISION Fail closed on privacy/authorization uncertainty.

VERIFICATION / EXIT GATE Fault injection proves each fallback preserves invariants.

Circuit breakers

TECHNICAL

Repeated planner/OCR failures or suspicious injection attempts stop automatic retries and require user/recovery path.

HUMAN EXPLANATION

If the same thing keeps failing, N-Eye does not hammer the system forever.

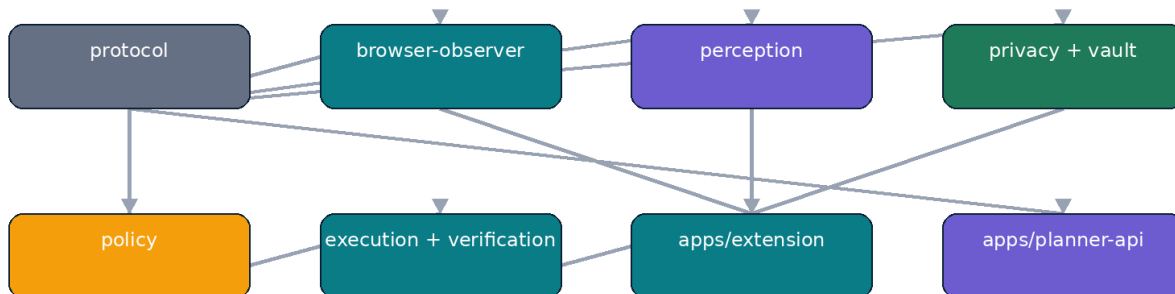
ARCHITECTURE DECISION Retry budgets are per failure class and action risk.

VERIFICATION / EXIT GATE Chaos tests verify bounded retries and no retry storms.

16. Repository & Package Architecture

One monorepo, explicit boundaries, no mystery folders.

Repository Dependency Direction



Monorepo spine

TECHNICAL

pnpm workspace for extension/shared TypeScript packages; Python planner isolated under apps/planner-api; test portal and benchmark harness co-located for reproducibility.

HUMAN EXPLANATION

Everything needed to build and prove N-Eye lives in one organized repository.

ARCHITECTURE DECISION Cross-language boundary is HTTP/JSON contract, not shared runtime imports.

VERIFICATION / EXIT GATE Fresh clone can install, test, build extension and start planner/test portal from documented commands.

Dependency direction

TECHNICAL

protocol lightweight; observer/perception/privacy/vault/policy/execution/verification/telemetry depend inward; apps compose packages; privacy/vault never import planner/network.

HUMAN EXPLANATION

Sensitive code must not know how to send data to the internet.

ARCHITECTURE DECISION No circular dependencies; egress is an application boundary.

VERIFICATION / EXIT GATE Dependency-lint/architecture test fails forbidden imports.

Folder minimalism

TECHNICAL

Create a package only when it owns a stable responsibility, has independent tests, or protects a trust boundary.

HUMAN EXPLANATION

We organize for clarity, not for a huge folder count.

ARCHITECTURE DECISION Start cohesive; split only when coupling/ownership evidence justifies it.

VERIFICATION / EXIT GATE Architecture review rejects empty/ceremonial packages.

17. Concrete Technology Stack

The on-ground stack for the 20-day prototype.

Browser client

TECHNICAL

Chrome Stable, Manifest V3, TypeScript strict mode, pnpm, Vite build tooling; React only for side-panel UI; content/service-worker logic remains framework-light.

HUMAN EXPLANATION

Use TypeScript for safety and Chrome-native APIs for the core. React helps the control panel, but we do not drag UI framework complexity into security-critical browser code.

ARCHITECTURE DECISION Exact package versions are pinned at implementation freeze after compatibility check.

VERIFICATION / EXIT GATE Clean build and extension load verified on target Chrome.

Validation/testing

TECHNICAL

Zod for local TypeScript runtime schemas where appropriate; Pydantic on FastAPI boundary; golden JSON contract fixtures prevent drift; Vitest for TS unit tests; pytest for planner; Playwright for controlled E2E/test portal.

HUMAN EXPLANATION

Both browser and server check the same shape of messages, and shared examples catch accidental disagreement.

ARCHITECTURE DECISION Protocol fixtures are canonical compatibility evidence across languages.

VERIFICATION / EXIT GATE Browser and planner contract tests consume identical golden fixtures.

Planner service

TECHNICAL

Python 3.12 environment, FastAPI, Pydantic, provider adapter, uvicorn locally; deployment only after local closed loop is stable.

HUMAN EXPLANATION

Python keeps the small AI gateway easy to inspect and test.

ARCHITECTURE DECISION No database required for core planner prototype.

VERIFICATION / EXIT GATE Stateless planner restart does not lose security-critical browser state.

Persistence

TECHNICAL

Memory-local token vault; minimal non-secret preferences in chrome.storage.local; session state only where MV3 lifecycle requires it; benchmark reports as files/artifacts.

HUMAN EXPLANATION

We do not add PostgreSQL or Redis just because enterprise systems often have them.

ARCHITECTURE DECISION Persist only what survives a privacy/lifecycle justification.

VERIFICATION / EXIT GATE Storage audit lists every persisted field and proves no secret class is persisted by default.

18. Detailed Folder Structure

Files should reveal architecture.

Root governance

TECHNICAL

AGENTS.md, README, package manifests, env example, CI, source-reference manifest, docs and ADRs live at root-level governance paths.

HUMAN EXPLANATION

Before coding, the repository itself tells humans and AI agents the rules.

ARCHITECTURE DECISION Governance files are version-controlled and reviewed like code.

VERIFICATION / EXIT GATE Agent can explain current architecture by reading repository docs without chat history.

Application folders

TECHNICAL

apps/extension contains background/content/sidepanel/offscreen-worker composition; apps/planner-api contains HTTP/provider boundary; apps/test-portal contains hostile synthetic scenarios.

HUMAN EXPLANATION

Three runnable things: browser extension, small AI server, and controlled fake websites for testing.

ARCHITECTURE DECISION Test portal is not production dependency.

VERIFICATION / EXIT GATE Production extension build has no test-portal dependency.

Core packages

TECHNICAL

packages/protocol, browser-observer, perception, privacy, vault, policy, execution, verification, telemetry; merge packages during bootstrap if boundaries are not yet justified, but preserve namespaces/contracts.

HUMAN EXPLANATION

Each folder owns one major job and cannot secretly take over another job.

ARCHITECTURE DECISION Architecture map and import rules stay synchronized.

VERIFICATION / EXIT GATE Automated architecture check validates forbidden dependency edges.

19. Protocol & Data Structures

Precise contracts make privacy enforceable.

Identifiers

TECHNICAL

Use branded/opaque TaskId, ElementId, PageEpoch, ActionId, TokenId at TypeScript boundaries to reduce accidental mixing.

HUMAN EXPLANATION

An element ID should not accidentally be used where a task ID belongs.

ARCHITECTURE DECISION Opaque identifiers are session/task scoped and contain no private meaning.

VERIFICATION / EXIT GATE Serialization tests show IDs reveal no raw private value.

Immutability

TECHNICAL

Raw observations may be assembled locally, but SafeContext and validated proposals are treated as immutable value objects after construction.

HUMAN EXPLANATION

Once the safe package is approved, later code cannot quietly add a secret to it.

ARCHITECTURE DECISION Builder creates final object; transport receives readonly validated representation.

VERIFICATION / EXIT GATE Mutation attempts do not alter serialized approved context.

Provenance

TECHNICAL

Observation/privacy findings carry source type, region/element reference and confidence without embedding unnecessary raw sensitive value in logs.

HUMAN EXPLANATION

When something is marked private, we can explain whether it came from a field type, pattern, OCR, or model.

ARCHITECTURE DECISION Provenance supports debugging and metrics without becoming a leakage channel.

VERIFICATION / EXIT GATE Telemetry snapshot contains classifications and IDs, not secret strings.

20. API Design

Small network surface, explicit contracts.

Planner endpoint

TECHNICAL

Prototype endpoint POST /v1/plan accepts versioned PlannerRequest{protocolVersion, SafeContext}; returns PlannerResponse{proposal, requestId, modelMetaSafe}.

HUMAN EXPLANATION

The browser sends one safe planning request and gets one structured suggestion back.

ARCHITECTURE DECISION No generic proxy endpoint, no arbitrary URL fetch, no raw upload endpoint in critical path.

VERIFICATION / EXIT GATE API contract tests reject unknown protocol versions and oversized bodies.

Authentication

TECHNICAL

Prototype uses server-side provider key and extension-to-planner auth appropriate to demo deployment; never embed provider secret in extension. Production auth is separate design.

HUMAN EXPLANATION

The expensive AI key stays on the server, not inside a downloadable browser extension.

ARCHITECTURE DECISION Do not pretend prototype auth equals enterprise identity.

VERIFICATION / EXIT GATE Build scan confirms no provider API key in extension bundle.

Logging

TECHNICAL

Planner logs request IDs, timings, safe schema metadata and sanitized failure codes; no raw-body analytics by default.

HUMAN EXPLANATION

Logs should tell us what failed without becoming a second copy of private browsing data.

ARCHITECTURE DECISION Privacy-safe observability is mandatory.

VERIFICATION / EXIT GATE Canary request produces zero canary hits in server logs.

21. Security Architecture

Defense in depth around an untrusted webpage and untrusted planner.

Least privilege

TECHNICAL

Minimize Chrome permissions, planner capabilities, message types and action vocabulary.

HUMAN EXPLANATION

N-Eye should have only the powers it actually needs.

ARCHITECTURE DECISION Every permission has documented reason; host permissions scoped for prototype/test needs.

VERIFICATION / EXIT GATE Permission audit before release.

Message validation

TECHNICAL

Every cross-context message has type/version/schema; privileged receiver validates sender/context and payload before action.

HUMAN EXPLANATION

A webpage or compromised content script cannot send random messages and expect the powerful part of the extension to obey.

ARCHITECTURE DECISION Treat content-script-originated data as untrusted input.

VERIFICATION / EXIT GATE Forged/unknown message fuzz tests fail closed.

CSP/dependencies

TECHNICAL

No remote code execution in extension; pin dependencies/lockfile; review new dependencies; generate SBOM if feasible; secret scanning and SAST in CI.

HUMAN EXPLANATION

We reduce supply-chain surprises and keep downloaded code from changing underneath us.

ARCHITECTURE DECISION Dependency additions require rationale in task report.

VERIFICATION / EXIT GATE CI blocks critical known vulnerabilities where tooling provides reliable signal.

22. Prompt Injection & Agent Safety

Web content cannot become authority.

Instruction/data separation

TECHNICAL

Planner prompt structurally labels system policy, user goal and webpage SafeContext; webpage text is explicitly untrusted data.

HUMAN EXPLANATION

The model is reminded that words from the webpage describe the page; they are not commands from the owner.

ARCHITECTURE DECISION Prompt wording is defense-in-depth; local validator is the real authority boundary.

VERIFICATION / EXIT GATE Injection page cannot expand action vocabulary or request secret reveal.

Action screening

TECHNICAL

Proposal checked against original task intent, local policy and current page; sensitive actions require confirmation.

HUMAN EXPLANATION

Even if the AI is tricked, its proposed action still has to match what the user actually asked for.

ARCHITECTURE DECISION Never rely on model self-policing for privileged action.

VERIFICATION / EXIT GATE Adversarial planner proposals are rejected locally.

Secret non-availability

TECHNICAL

Best defense against exfiltration is that remote model never receives raw secrets/token mappings in the first place.

HUMAN EXPLANATION

An AI cannot reveal a password it never saw.

ARCHITECTURE DECISION Data minimization precedes prompt-injection mitigation.

VERIFICATION / EXIT GATE Canary secrets remain absent even when page explicitly asks model to reveal them.

23. Testing Architecture

Testing is a parallel engineering system, not a final phase.

Static/unit/contract

TECHNICAL

Typecheck, lint, unit tests, schema/property tests, cross-language contract fixtures and architecture dependency checks run continuously.

HUMAN EXPLANATION

Fast tests catch mistakes before we even open Chrome.

ARCHITECTURE DECISION Security invariants get tests, not comments.

VERIFICATION / EXIT GATE Every task defines tests before implementation.

Integration/E2E

TECHNICAL

Real extension loaded against controlled test portal; Playwright drives scenarios but does not replace product executor.

HUMAN EXPLANATION

We test the real extension in a real browser against fake sites designed to break it.

ARCHITECTURE DECISION E2E asserts network payload, local action and post-state.

VERIFICATION / EXIT GATE CI/local harness reproduces core scenarios.

Adversarial/privacy

TECHNICAL

Canaries in DOM, hidden DOM, fields, OCR images, prompt injections, logs and token misuse scenarios.

HUMAN EXPLANATION

We deliberately try to make secrets escape and fail the build if they do.

ARCHITECTURE DECISION Leakage test searches actual bytes and storage surfaces.

VERIFICATION / EXIT GATE Zero forbidden canary occurrence outside approved boundary.

Performance/generalization

TECHNICAL

Held-out pages, p50/p95 stage latency, payload size,

HUMAN EXPLANATION

We measure speed and resource use on pages the

capture/OCR frequency, model/runtime size, memory proxy, CPU/GPU path.

system was not tuned to memorize.

ARCHITECTURE DECISION Only actual target-hardware measurements are RESULTS.

VERIFICATION / EXIT GATE Benchmark report records environment and commit hash.

24. Synthetic Test Laboratory

The hostile world we control.

Scenario suite

TECHNICAL

A basic form; mixed private/contextual page; pixel-text image; canvas target; dynamic SPA; prompt injection; high-risk action; PDF-like viewer; hidden leak; accessibility leak; failure path.

HUMAN EXPLANATION

Instead of waiting for random websites to expose bugs, we build websites that intentionally contain each difficult case.

ARCHITECTURE DECISION Fixtures are deterministic and versioned.

VERIFICATION / EXIT GATE Each architecture claim maps to at least one scenario.

Hold-outs

TECHNICAL

Keep variants/templates hidden from tuning; no hard-coded selectors or fixture IDs in product code.

HUMAN EXPLANATION

The final test includes pages the agent has not practiced on.

ARCHITECTURE DECISION Generalization is mechanism-based, not memorization.

VERIFICATION / EXIT GATE Static scan and hidden suite detect demo-specific selectors.

Fault injection

TECHNICAL

Simulate planner timeout, malformed output, worker crash, OCR failure, stale page, network loss and verification ambiguity.

HUMAN EXPLANATION

We intentionally break pieces to prove N-Eye fails safely.

ARCHITECTURE DECISION Recovery behavior is part of acceptance criteria.

VERIFICATION / EXIT GATE Fault suite never causes privacy/authority invariant violation.

25. Development Environment

Reproducibility starts before features.

Toolchain

TECHNICAL

Git/private GitHub, Node LTS, pnpm, Python 3.12 env, Chrome Stable/DevTools, Cursor, Codex, Playwright, lint/format/typecheck/test tooling.

HUMAN EXPLANATION

Every developer and AI agent works from the same predictable toolbox.

ARCHITECTURE DECISION Pin lockfiles; document versions at release candidate.

VERIFICATION / EXIT GATE Fresh machine bootstrap follows runbook successfully.

Branch discipline

TECHNICAL

One bounded feature per branch/issue; one implementation agent at a time; second agent reviews; human merges.

HUMAN EXPLANATION

Cursor and Codex do not edit the same feature simultaneously and create chaos.

ARCHITECTURE DECISION Git history is the source of truth, not chat.

VERIFICATION / EXIT GATE Each merged task has acceptance evidence and review summary.

Secrets

TECHNICAL

Only .env.example committed; real provider keys in local/deployment secret mechanism; no secrets in prompts, screenshots, fixtures or logs.

HUMAN EXPLANATION

Real credentials never become test data.

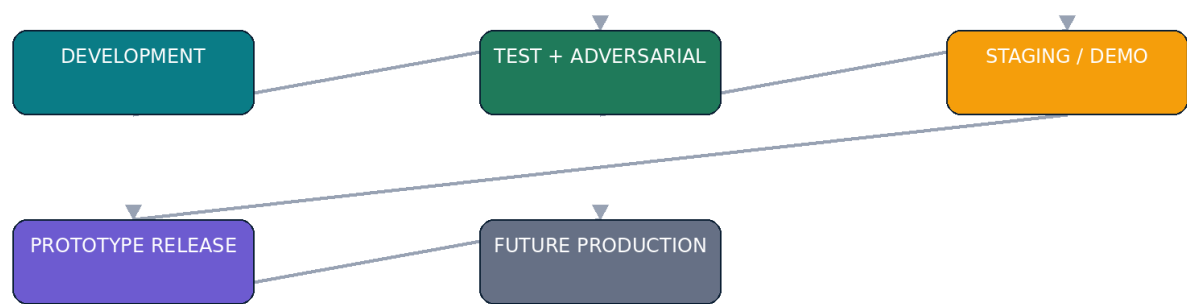
ARCHITECTURE DECISION Synthetic canaries only.

VERIFICATION / EXIT GATE Secret scanner passes before merge/release.

26. Development -> Test -> Staging -> Release

One architecture, progressively stronger evidence.

Engineering Delivery Pipeline



Development

TECHNICAL

Fast local loop with mock planner by default, synthetic data, verbose privacy-safe diagnostics and bounded feature tests.

HUMAN EXPLANATION

Build one piece quickly without needing the internet or real accounts.

ARCHITECTURE DECISION Mock and real planner share contracts.

VERIFICATION / EXIT GATE Feature branch cannot merge until local acceptance passes.

Test

TECHNICAL

Automated suites, adversarial pages, canary scans, contract tests, fault injection and benchmark smoke tests.

HUMAN EXPLANATION

Now we try to break what we built.

ARCHITECTURE DECISION Test environment may enable extra assertions but not weaker security.

VERIFICATION / EXIT GATE Required suite green on exact commit.

Staging/demo

TECHNICAL

Packaged extension + deployed/local stable planner + deterministic test portal + frozen model assets + evidence mode + backup paths.

HUMAN EXPLANATION

This is the version we rehearse exactly like competition day.

ARCHITECTURE DECISION No experimental dependencies after freeze without rollback plan.

VERIFICATION / EXIT GATE 10 consecutive clean demo runs.

Prototype release

TECHNICAL

Tagged commit, checksums/artifacts, evidence report, known limitations, setup/runbook and backup video; not labeled production-grade.

HUMAN EXPLANATION

We can reproduce exactly what was shown and explain what is still unfinished.

ARCHITECTURE DECISION Honest release labeling.

VERIFICATION / EXIT GATE Fresh clone/build reproduces core proof.

27. CI/CD & Quality Gates

Automation enforces what documentation promises.

Pull-request gate

TECHNICAL

Format/lint/typecheck/unit/contract/architecture checks, secret scan, planner tests and selected E2E.

HUMAN EXPLANATION

Bad code should fail before a human wastes time reviewing it.

ARCHITECTURE DECISION Security-sensitive suites cannot be skipped silently.

VERIFICATION / EXIT GATE Protected branch requires green checks.

Nightly/deep gate

TECHNICAL

Full browser scenarios, canary suite, hidden variants, fault injection and benchmark trend where infrastructure permits.

HUMAN EXPLANATION

Slow expensive tests run regularly without blocking every tiny edit.

ARCHITECTURE DECISION Performance regressions are flagged, not automatically “optimized” by relaxing privacy.

VERIFICATION / EXIT GATE Trend report links commit to metrics.

Release gate

TECHNICAL

Clean install, packaged extension, permissions/CSP review, dependency audit, demo rehearsal, evidence integrity and rollback package.

HUMAN EXPLANATION

Before competition, we test the exact package we will show, not a developer-only version.

ARCHITECTURE DECISION Release is immutable once signed/tagged for demo except emergency patch process.

VERIFICATION / EXIT GATE Release checklist signed off by human owner.

28. Observability & Telemetry

Measure the system without recreating the privacy problem.

Stage tracing

TECHNICAL

Record task/action IDs, stage start/end, safe outcome codes, sensor path and aggregate sizes; never raw private values.

HUMAN EXPLANATION

We know where time is spent without recording what the user typed.

ARCHITECTURE DECISION Telemetry schema is allowlisted.

VERIFICATION / EXIT GATE Canary values never appear in traces.

Metrics

TECHNICAL

Latency p50/p95, payload bytes, OCR/capture count, planner time, validation time, verification rate, retry counts, memory proxy and privacy benchmark metrics.

HUMAN EXPLANATION

We can prove whether N-Eye is fast, small and reliable instead of guessing.

ARCHITECTURE DECISION Metric names/units/environment documented.

VERIFICATION / EXIT GATE Benchmark report reproducible from commit and scenario set.

Debug mode

TECHNICAL

Local-only evidence inspector shows sanitized SafeContext, proposal, validation reason and verification result; secret fields never displayed.

HUMAN EXPLANATION

Developers and judges can see the mechanism without seeing the private values.

ARCHITECTURE DECISION Debug UI is not a security boundary.

VERIFICATION / EXIT GATE Privacy red-team includes debug surfaces.

29. Performance Architecture

Optimization follows measurement and preserves safety.

Critical path

TECHNICAL

Avoid continuous screenshots/VLM; cache only safe reusable metadata; use ROI capture; workerize OCR; parallelize independent safe steps carefully; cap context size.

HUMAN EXPLANATION

The fastest work is work we do not need to do.

ARCHITECTURE DECISION Never trade privacy correctness for latency without explicit decision.

VERIFICATION / EXIT GATE Profiler identifies stage bottleneck before optimization.

Budgets

TECHNICAL

Set engineering targets for observation, privacy, serialization, planner and verification, but label TARGET until measured.

HUMAN EXPLANATION

We can aim for speed without pretending the target is already achieved.

ARCHITECTURE DECISION Targets are hardware/scenario dependent.

VERIFICATION / EXIT GATE Result tables include machine, Chrome, commit and dataset.

Backpressure

TECHNICAL

Limit concurrent perception/planner work; cancel stale requests when page epoch changes; debounce noisy mutations.

HUMAN EXPLANATION

A constantly changing webpage should not cause hundreds of expensive jobs.

ARCHITECTURE DECISION Newest valid page state supersedes stale work.

VERIFICATION / EXIT GATE Mutation storm test stays bounded and executes no stale action.

30. Data Science & Evaluation

Evidence quality matters as much as code quality.

Dataset design

TECHNICAL

Synthetic labeled corpus covers privacy classes, OCR text, visual-only controls, benign negatives, ambiguous context and hidden hold-outs.

HUMAN EXPLANATION

We need examples with known answers so accuracy numbers mean something.

ARCHITECTURE DECISION Separate train/tune fixtures from hold-out evaluation.

VERIFICATION / EXIT GATE Dataset manifest records provenance and labels.

Metrics

TECHNICAL

Privacy detection uses per-class precision/recall/F1; redaction uses residual leakage + region precision/utility; visual path uses target/text accuracy; system uses task success and verification.

HUMAN EXPLANATION

One overall “accuracy” number can hide serious failures, so we measure each kind of mistake separately.

ARCHITECTURE DECISION Report confusion cases and class imbalance.

VERIFICATION / EXIT GATE Metrics script deterministic on fixed dataset.

Experiment registry

TECHNICAL

Every model/runtime/threshold experiment records hypothesis, variables, hardware, commit, dataset, result and decision.

HUMAN EXPLANATION

We remember why we chose an OCR/model setting instead of repeatedly guessing.

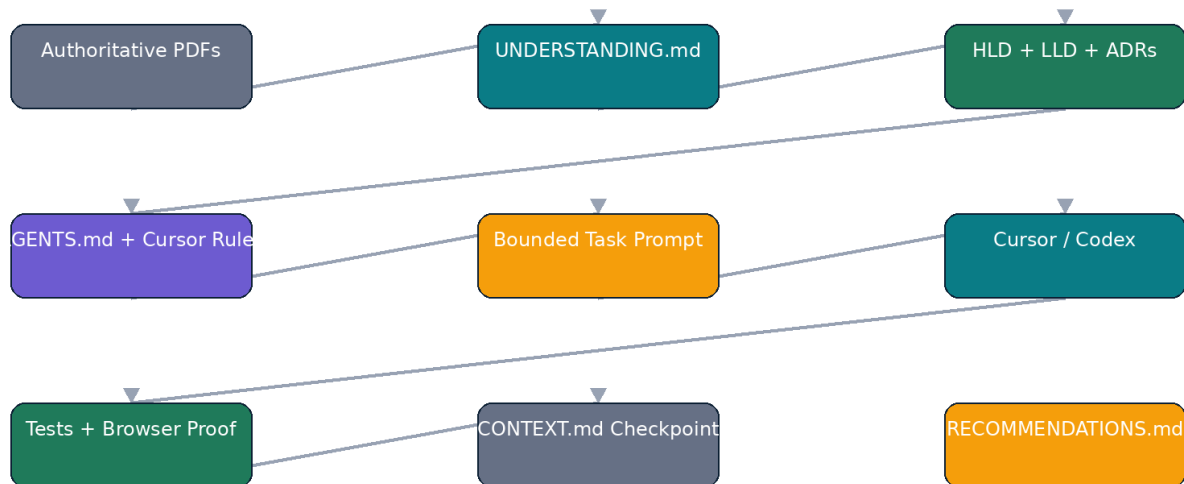
ARCHITECTURE DECISION Experiments do not silently become architecture.

VERIFICATION / EXIT GATE Accepted experiment decision links to ADR/config change.

31. AI Engineering Control Plane

Cursor/Codex are accelerators, not architecture owners.

AI Engineering Control Plane



Repository constitution

TECHNICAL

Root AGENTS.md states product, invariants, required reading, forbidden changes, testing/reporting rules; scoped Cursor .mdc rules add subsystem-specific constraints.

HUMAN EXPLANATION

Every AI coding session starts with the same house rules.

ARCHITECTURE DECISION Rules are short enough to remain usable and specific enough to enforce boundaries.

VERIFICATION / EXIT GATE Agent can state applicable rules before edit.

Prompt contract

TECHNICAL

Every task prompt defines current state, objective, non-objectives, allowed scope, contracts, invariants, tests, verification, stop conditions and final report.

HUMAN EXPLANATION

We never tell an AI “build N-Eye.” We give it one carefully bounded engineering job.

ARCHITECTURE DECISION Prompt is a mini-specification, not motivational prose.

VERIFICATION / EXIT GATE Agent stops rather than inventing missing architecture.

Independent review

TECHNICAL

One agent implements; the other reviews privacy/security/edge cases/tests/diff. Human approves architecture and merge.

HUMAN EXPLANATION

The coder does not grade its own exam.

ARCHITECTURE DECISION Reviewer receives acceptance criteria and threat assumptions, not just diff.

VERIFICATION / EXIT GATE Confirmed review findings fixed and suite rerun before merge.

32. Project Memory & Governance Files

The repository remembers what chat forgets.

UNDERSTANDING.md

TECHNICAL

Stable explanation of product, scope, mental model, terminology, non-goals and canonical loop.

HUMAN EXPLANATION

A new engineer can understand N-Eye without reading every chat.

ARCHITECTURE DECISION Changes only when product understanding changes.

VERIFICATION / EXIT GATE Onboarding review can restate architecture correctly.

CONTEXT.md

TECHNICAL

Operational checkpoint: current architecture version, completed tasks, current gate, next task, defects, risks, ADRs, experiments, measured results and pending recommendations.

HUMAN EXPLANATION

This is the project’s short-term memory.

ARCHITECTURE DECISION Refresh every 10 completed tasks or major milestone/ADR, whichever comes first.

VERIFICATION / EXIT GATE New agent resumes from context without guessing.

RECOMMENDATIONS.md

TECHNICAL

Unapproved improvements with evidence, benefits, costs, security/privacy/performance/migration impact and urgency.

HUMAN EXPLANATION

AI tools put “maybe better” ideas in a waiting room instead of changing the project by themselves.

ARCHITECTURE DECISION Accepted recommendation becomes ADR/task; rejected one records rationale.

VERIFICATION / EXIT GATE No recommendation is implemented without approval.

33. ADR System

Decisions must be explainable later.

ADR format

TECHNICAL

Context, decision, status, alternatives, consequences, security/privacy impact, performance impact, migration/rollback, evidence and approver.

HUMAN EXPLANATION

Months later we can answer “why did we choose this?”

ARCHITECTURE DECISION ADRs are immutable history; supersede rather than rewrite past decisions.

VERIFICATION / EXIT GATE Architecture docs link active ADRs.

ADR triggers

TECHNICAL

Protocol/schema, trust boundary, persistence, dependency, browser permission, remote egress, model/runtime critical path, risk policy and deployment architecture changes.

HUMAN EXPLANATION

Small code cleanup does not need bureaucracy; decisions that can change safety or system shape do.

ARCHITECTURE DECISION Agents must classify whether task triggers ADR.

VERIFICATION / EXIT GATE PR template requires architecture-impact declaration.

Recommendation path

TECHNICAL

Discover -> document -> human review -> accept/reject -> ADR if accepted -> task -> implementation -> verification.

HUMAN EXPLANATION

Good ideas are encouraged without allowing uncontrolled architecture drift.

ARCHITECTURE DECISION Urgent security fix can use expedited ADR with postmortem.

VERIFICATION / EXIT GATE Audit trail links recommendation to final decision.

34. Task Graph & 20-Day Execution Model

Deadline is time; task gates are the unit of progress.

Task-based flow

TECHNICAL

T00 architecture freeze through repository/protocol/runtime/privacy/planner/perception/robustness/benchmark/release gates. Move immediately when exit gate passes; do not wait for calendar day.

HUMAN EXPLANATION

If we finish today’s task early, we start the next one immediately.

ARCHITECTURE DECISION Dependencies, not dates, control order.

VERIFICATION / EXIT GATE No dependent task starts while prerequisite invariant is broken.

Definition of done

TECHNICAL

Each task has preconditions, required reading, implementation scope, tests, browser proof, privacy/security checks, independent review, evidence and context update rule.

HUMAN EXPLANATION

A task is either proven done or still open.

ARCHITECTURE DECISION “Mostly done” cannot close a gate.

VERIFICATION / EXIT GATE Task report includes exact commands/results and residual risks.

Critical path

TECHNICAL

First closed deterministic loop before OCR/model sophistication; real planner after mock/local authority; visual path after privacy boundary; benchmarks before final claims.

HUMAN EXPLANATION

We prove the plumbing before adding intelligence.

ARCHITECTURE DECISION This sequence is preserved unless evidence-backed ADR changes it.

VERIFICATION / EXIT GATE By early milestone, mock end-to-end loop works with zero real planner dependency.

35. 40-Task Implementation Backlog

A complete sequence from empty repo to evidence pack.

Foundation tasks

TECHNICAL

T00 Architecture Freeze; T01 Repository Constitution;
T02 Toolchain/CI; T03 Protocol Foundation; T04 MV3
Runtime; T05 Typed Messaging; T06 Test Portal Base.

HUMAN EXPLANATION

First we create the rules, skeleton and communication pipes.

ARCHITECTURE DECISION No ML dependencies yet.

VERIFICATION / EXIT GATE Extension loads and all baseline tests pass.

Core trust-loop tasks

TECHNICAL

T07 Observer; T08 Registry/PageEpoch; T09 Privacy
Detection; T10 Policy; T11 Sanitizer; T12 Vault; T13
SafeContext; T14 Egress Guard; T15 Mock Planner;
T16 Validator; T17 Re-ground; T18 Executor; T19
Verifier; T20 Closed Loop.

HUMAN EXPLANATION

Then we build the complete safe loop without
depending on AI intelligence.

ARCHITECTURE DECISION This is the highest-priority milestone.

VERIFICATION / EXIT GATE Private form -> protected context -> mock proposal -> local action -> verified state change.

Intelligence/robustness tasks

TECHNICAL

T21 Planner API; T22 Real Planner; T23 ROI; T24 OCR;
T25 OCR Privacy; T26 Visual Grounding; T27 Adaptive
Controller; T28 Risk/Confirmation; T29 Injection
Defense; T30 SPA/Stale; T31 Recovery; T32
WebGPU/WASM experiment.

HUMAN EXPLANATION

Only after the safe loop works do we add cloud
reasoning and harder visual cases.

ARCHITECTURE DECISION Each expensive component must justify itself with a failing scenario/benchmark.

VERIFICATION / EXIT GATE Visual-only and hostile-page scenarios pass without boundary violation.

Evidence/release tasks

TECHNICAL

T33 Benchmark Harness; T34 Privacy Red Team; T35 Held-Out Generalization; T36 Performance Profile; T37 Evidence UI; T38 Packaging/Staging Freeze; T39 Release/Evidence Pack.

HUMAN EXPLANATION

Finally we prove, freeze, rehearse and package.

ARCHITECTURE DECISION No new architecture after freeze except emergency fix.

VERIFICATION / EXIT GATE Fresh clone and 10x deterministic demo pass.

36. Master Prompt Standard

How every coding task will be fed to Cursor/Codex.

Prompt anatomy

TECHNICAL

ROLE, AUTHORITATIVE SOURCES, CURRENT STATE, ARCH VERSION, TASK, WHY, PRECONDITIONS, OBJECTIVE, NON-OBJECTIVES, ALLOWED/FORBIDDEN SCOPE, RUNTIME/TRUST, INPUT/OUTPUT, LIFETIMES, PRIVACY/SECURITY/PERF, DEPENDENCIES, IMPLEMENTATION, ERROR/FALLBACK, TESTS, MANUAL PROOF, ACCEPTANCE, STOP, SELF-REVIEW, FINAL REPORT, CONTEXT/RECOMMENDATION RULE.

HUMAN EXPLANATION

The AI gets a complete engineering brief, not a vague request.

ARCHITECTURE DECISION Prompts remain bounded to one task even when detailed.

VERIFICATION / EXIT GATE Agent’s plan maps every acceptance criterion to code/test evidence.

Required source reading

TECHNICAL

Genesis reads all authoritative PDFs and creates distilled governance docs. Later tasks read AGENTS/CONTEXT/relevant HLD/LLD/ADRs; re-open source PDFs for architecture/privacy/protocol/perception/SIH-claim changes.

HUMAN EXPLANATION

We do not waste every tiny task rereading hundreds of pages, but we force source re-check whenever a critical decision is touched.

ARCHITECTURE DECISION Source provenance beats stale chat memory.

VERIFICATION / EXIT GATE Task report lists sources actually consulted.

Automatic bug repair

TECHNICAL

Agent fixes defects introduced by current task within allowed scope; architecture/schema/dependency/trust changes become blocker/recommendation/ADR, not silent fixes.

HUMAN EXPLANATION

The AI cleans up its own mess, but it cannot use “bug fix” as permission to redesign the system.

ARCHITECTURE DECISION Regression suite reruns after repair.

VERIFICATION / EXIT GATE Final report lists bugs found, fixes, tests and unresolved risks.

37. Release Evidence & SIH Demo

The demo shows proof, not marketing claims.

90-second proof

TECHNICAL

Private controlled page -> local detect/OCR -> actual network payload with secret absent -> structured planner proposal -> local validation/re-ground/token -> action -> verified state -> metrics.

HUMAN EXPLANATION

Judges see the private information, then see that it did not leave, then see AI still complete the task safely.

ARCHITECTURE DECISION DevTools/network evidence outranks a “protected” badge.

VERIFICATION / EXIT GATE Demo script completes repeatedly from clean state.

Evidence pack

TECHNICAL

Architecture diagram, threat model, canary report, PII metrics, redaction results, visual hold-out results, latency/resource profile, test matrix, known limitations, commit/build IDs.

HUMAN EXPLANATION

Every major claim has a document or measurement behind it.

ARCHITECTURE DECISION Unsupported claims are removed.

VERIFICATION / EXIT GATE Evidence artifacts reproducible from tagged commit.

Backup strategy

TECHNICAL

Mock planner same contract, WASM/CPU fallback, packaged model assets, controlled portal, sanitized request inspector, recorded backup video, shortened trust-boundary proof.

HUMAN EXPLANATION

If internet/GPU/provider/demo conditions fail, we can still prove the architecture honestly.

ARCHITECTURE DECISION Backup must disclose what it proves and does not prove.

VERIFICATION / EXIT GATE Rehearse each backup path before freeze.

38. Future Company Evolution

Scale without corrupting prototype architecture.

Production hardening

TECHNICAL

Enterprise identity/tenant isolation, managed policies, hardened local secret lifecycle, signed model delivery, audit controls, regional processing, update channels, multi-browser QA and security/compliance programs.

HUMAN EXPLANATION

If N-Eye becomes a company, these are the systems that turn a prototype into trusted infrastructure.

ARCHITECTURE DECISION These are future seams, not current claims.

VERIFICATION / EXIT GATE Prototype APIs avoid choices that make later hardening impossible.

Model independence

TECHNICAL

Planner/provider abstraction and SafeContext/ActionProposal contracts keep core trust model independent of one model vendor.

HUMAN EXPLANATION

N-Eye's value should not disappear when a better AI model arrives.

ARCHITECTURE DECISION Do not expose provider-specific semantics to browser authority layer.

VERIFICATION / EXIT GATE Swap provider in staging without changing local validator/executor.

Platform direction

TECHNICAL

Browser client can evolve toward SDK/runtime trust layer for other agents while preserving minimum disclosure + constrained authority + verification.

HUMAN EXPLANATION

The browser is the first vehicle, not the final company definition.

ARCHITECTURE DECISION Company evolution requires new threat models/ADRs, not assumption that prototype controls automatically scale.

VERIFICATION / EXIT GATE Future roadmap distinguishes validated customer needs from speculative features.

Appendix A — Proposed Repository Tree

Implementation-oriented structure; packages may be merged during bootstrap if boundaries are not yet justified.

```
n-eye/
├── AGENTS.md
├── README.md
├── package.json
├── pnpm-workspace.yaml
├── .gitignore
├── .env.example
├── .github/workflows/ci.yml
├── .cursor/rules/
│   ├── architecture.mdc
│   ├── privacy.mdc
│   ├── security.mdc
│   ├── testing.mdc
│   ├── extension.mdc
│   └── planner.mdc
├── reference/
│   ├── SOURCE_MANIFEST.md
│   └── [authoritative N-Eye PDFs/PPTX]
├── apps/
│   ├── extension/
│   │   ├── src/background/
│   │   ├── src/content/
│   │   ├── src/sidepanel/
│   │   ├── src/workers/
│   │   └── tests/
│   ├── planner-api/
│   │   ├── app/api/
│   │   ├── app/domain/
│   │   ├── app/providers/
│   │   └── tests/
│   └── test-portal/
│       ├── scenarios/
│       └── hidden/
├── packages/
│   ├── protocol/
│   ├── browser-observer/
│   ├── perception/
│   ├── privacy/
│   ├── vault/
│   ├── policy/
│   ├── execution/
│   ├── verification/
│   └── telemetry/
├── bench/
│   ├── datasets/
│   ├── scenarios/
│   ├── hidden/
│   ├── experiments/
│   └── reports/
└── docs/
    ├── UNDERSTANDING.md
    ├── CONTEXT.md
    ├── HLD.md
    ├── LLD.md
    ├── ARCHITECTURE.md
    ├── TRUST-MODEL.md
    ├── PRIVACY-BOUNDARY.md
    ├── THREAT-MODEL.md
    ├── PROTOCOLS.md
    ├── TEST-STRATEGY.md
    ├── RELEASE-STRATEGY.md
    ├── RUNBOOK.md
    ├── RECOMMENDATIONS.md
    └── decisions/ADR-xxx-*.md
```

Appendix B — Core Contract Matrix

Every important object has an owner, lifetime, serialization rule and trust classification.

Object	Owner	Lifetime	Network?	Trust
RawElement	Observer	page epoch	No	Untrusted observation
ElementRegistryEntry	Content/local	page epoch	No	Local sensitive reference
RawScene	Local fusion	task/page epoch	Never	Local sensitive
PrivacyFinding	Privacy engine	task	Local only by default	Local sensitive metadata
TokenBinding	Vault	task/expiry	Never	Highest sensitivity
SafeContext	Builder	planning turn	Yes	Approved outbound
ActionProposal	Planner	planning turn	Yes inbound	Untrusted
ValidatedAction	Validator	single action	Local only	Trusted local authority
ExecutionResult	Executor	action	Local/safe summary	Trusted observation
VerificationResult	Verifier	action/task	Safe summary	Trusted evaluator
StageMetric	Telemetry	session/report	Aggregate only	Privacy-safe

Appendix C — Action Vocabulary & Risk Baseline

Initial bounded vocabulary; exact policy thresholds require implementation testing.

Action	Parameters	Typical Risk	Local Gate
CLICK	targetId	Low-Medium	Target current + visible + permitted
TYPE_TOKEN	targetId + tokenId	Medium-High	Token scope + target semantics + origin + expiry
SCROLL	direction + amount	Low	Bounds + active page
SELECT	targetId + safe option	Medium	Current select + option allowed
WAIT	bounded ms	Low	Maximum duration
ASK_USER	safe reason	None	No private echo
COMPLETE	safe summary	None	Verifier/task state supports completion

FORBIDDEN Arbitrary JavaScript, shell commands, unrestricted CSS/XPath selectors, raw secret requests, unrestricted coordinates, generic browser APIs, or any planner-defined executable payload.

Appendix D — Failure & Fallback Matrix

The safest fallback is the one that preserves authority and privacy invariants.

Failure	Safe fallback	Forbidden recovery
DOM insufficient	Targeted ROI OCR	No raw screenshot cloud fallback
OCR unavailable/slow	Smaller ROI -> WASM/CPU -> ask user	Do not disable privacy
Local CV unavailable	OCR/structure alternate -> ask user	Do not grant planner coordinates
Planner timeout	Bounded retry -> mock for demo / ask user	Do not send more raw context
Malformed proposal	Reject -> replan/ask	Do not coerce into executable action
Stale target	Re-observe -> replan	Never click stale coordinate
Token mismatch	Deny -> ask user	Never reveal/force token
Verification ambiguous	Re-observe -> replan/ask	Never mark success
Repeated suspicious injection	Circuit break -> safe stop	Never let model expand privileges

Appendix E — Test Matrix

Minimum proof suite mapped to architecture claims.

Scenario	Subsystems	Pass evidence
Basic form	Observer/privacy/vault/action	Email/phone protected; password/OTP never sent; verified Continue
Pixel text	OCR + privacy	Image PII detected locally; outbound safe
Canvas control	Visual grounding	Genuine pixel-only target resolved locally
Dynamic SPA	Epoch/re-ground	Stale proposal blocked
Prompt injection	Planner/local authority	Injected page instruction cannot reveal secret or bypass policy
High-risk action	Risk/confirmation	Submit/Delete/Upload gated
Hidden leak	Observer	Hidden DOM/data attributes absent
Accessibility leak	Observer/privacy	Sensitive semantics do not leak via ARIA/title
Network failure	Recovery	No privacy relaxation; safe failure
Worker/GPU failure	Fallback	Correctness path survives without unsafe escalation

Appendix F — Task Definition of Done

Template used for every implementation task.

1. Preconditions satisfied and required sources read.
2. Agent states exact files to change and why.
3. Architecture/trust/privacy/security impact classified.
4. Implementation remains inside allowed scope.
5. Static checks pass.
6. Unit/contract/property tests pass.
7. Relevant integration/E2E tests pass.
8. Privacy canary/security tests pass where applicable.
9. Real Chrome/manual verification completed where applicable.
10. Independent review completed and confirmed findings fixed.
11. Regression suite rerun.
12. Evidence artifact/report created.
13. Residual risks and limitations stated.
14. CONTEXT.md updated if checkpoint condition reached.
15. Human gate approves merge when architecture/security-sensitive.

Appendix G — Agent Prompt Skeleton

Copy-paste structure; each real task fills every applicable field.

ROLE <task-specific, explicit, bounded content>

AUTHORITATIVE SOURCES <task-specific, explicit, bounded content>

CURRENT REPOSITORY STATE <task-specific, explicit, bounded content>

CURRENT ARCHITECTURE VERSION <task-specific, explicit, bounded content>

CURRENT TASK <task-specific, explicit, bounded content>

WHY THIS EXISTS <task-specific, explicit, bounded content>

PRECONDITIONS <task-specific, explicit, bounded content>

OBJECTIVE <task-specific, explicit, bounded content>

NON-OBJECTIVES <task-specific, explicit, bounded content>

ALLOWED SCOPE <task-specific, explicit, bounded content>

FORBIDDEN CHANGES <task-specific, explicit, bounded content>

RUNTIME LOCATION <task-specific, explicit, bounded content>

TRUST CLASSIFICATION <task-specific, explicit, bounded content>

INPUT CONTRACT <task-specific, explicit, bounded content>

OUTPUT CONTRACT <task-specific, explicit, bounded content>

DATA LIFETIME <task-specific, explicit, bounded content>

PRIVACY INVARIANTS <task-specific, explicit, bounded content>

SECURITY INVARIANTS <task-specific, explicit, bounded content>

PERFORMANCE CONSTRAINTS <task-specific, explicit, bounded content>

DEPENDENCY RULES <task-specific, explicit, bounded content>

IMPLEMENTATION PROCESS <task-specific, explicit, bounded content>

ERROR HANDLING <task-specific, explicit, bounded content>

FALLBACK BEHAVIOR <task-specific, explicit, bounded content>

UNIT TESTS <task-specific, explicit, bounded content>

PROPERTY/CONTRACT TESTS <task-specific, explicit, bounded content>

INTEGRATION/E2E TESTS <task-specific, explicit, bounded content>

ADVERSARIAL TESTS <task-specific, explicit, bounded content>

PRIVACY CANARY TESTS <task-specific, explicit, bounded content>

MANUAL CHROME TEST <task-specific, explicit, bounded content>

ACCEPTANCE CRITERIA <task-specific, explicit, bounded content>

STOP CONDITIONS <task-specific, explicit, bounded content>

SELF-REVIEW <task-specific, explicit, bounded content>

INDEPENDENT REVIEW HANDOFF <task-specific, explicit, bounded content>

FINAL REPORT FORMAT <task-specific, explicit, bounded content>

CONTEXT UPDATE RULE <task-specific, explicit, bounded content>

RECOMMENDATION RULE <task-specific, explicit, bounded content>

Appendix H — Governance File Contracts

What each Markdown file is allowed to contain.

File	Purpose	Update cadence
AGENTS.md	Non-negotiable engineering behavior and global invariants	Frequent but reviewed
UNDERSTANDING.md	Stable product mental model, scope, glossary, non-goals	Rare
CONTEXT.md	Current operational checkpoint and project memory	Every 10 tasks/milestone/ADR
HLD.md	System containers, trust zones, data/control flows, deployment	Architecture changes
LLD.md	Modules, interfaces, state machines, schemas, algorithms	Design changes
THREAT-MODEL.md	Assets, attackers, trust boundaries, threats, mitigations, tests	Security milestones
PROTOCOLS.md	RawScene/SafeContext/ActionProposal/message contracts	Protocol changes
TEST-STRATEGY.md	Test pyramid, fixtures, canaries, gates, benchmarks	Testing changes
RECOMMENDATIONS.md	Unapproved ideas with evidence/impact	As discovered
decisions/ADR-*	Accepted/rejected architecture decisions	Per decision

Appendix I — Verification Classification

How claims are labeled before they enter code, docs or presentation.

Label	Meaning	Example
SOURCE FACT	Supported by authoritative N-Eye source	Architecture invariant from 95-page masterbook
VERIFIED EXTERNAL FACT	Checked against authoritative current documentation	MV3 service-worker/offscreen/rules behavior
ARCHITECTURE DECISION	Chosen design, with rationale and alternatives	FastAPI planner; memory-local prototype vault
EXPERIMENT REQUIRED	Cannot be known honestly before measurement	OCR model/runtime, WebGPU benefit, latency budgets
RESULT	Measured on exact build/hardware/dataset	p95 latency, PII F1, leakage count

RULE Never promote TARGET, literature evidence, or architecture expectation into RESULT.

Appendix J — Source & Verification Register

Internal sources define N-Eye; external sources verify changing platform/security facts.

Primary internal source

- N-Eye_SIH2026_Human_First_Engineering_Masterbook_FINAL.pdf — 95-page primary engineering authority.

Supporting internal sources

- N-Eye_context.md — continuity and current decisions.
- N-Eye_Human_First_Ultimate_Engineering_Masterbook_v2.pdf — real-life build guide.
- N-Eye-Prototype(Instant).pdf — execution blueprint.
- N-Eye_SIH2026_Prototype_Blueprint_FINAL.pptx and N-Eye_Prototype_Blueprint_FINAL.pptx — judge-facing architecture summaries.
- N-EyeDeep.pdf — research/architecture decision space.
- N-Eye-Company.pdf — long-term product/company direction.

External verification used for architecture v1.0

- Chrome for Developers — Chrome Extensions / Manifest V3 documentation, including service workers, side panel, offscreen and extension security guidance.
- Cursor Docs — Project Rules and AGENTS.md behavior.
- OWASP GenAI / Cheat Sheet Series — prompt injection and AI-agent security mitigations.
- NIST SP 800-218 SSDF — secure software development practices integrated into the SDLC.

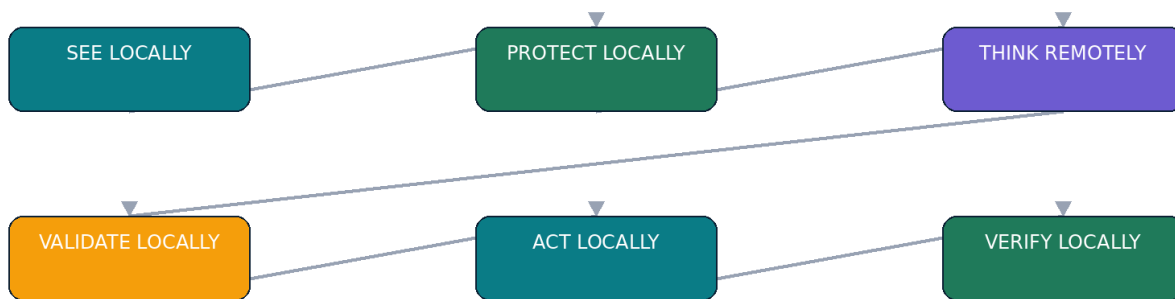
RECHECK AT FREEZE Exact dependency versions, Chrome API behavior, model/runtime compatibility, Cursor/Codex behavior, and deployment constraints must be reverified at implementation/release freeze.

Final Architecture Constitution

These rules survive every implementation task unless an approved ADR supersedes them.

16. Raw passwords, OTPs, API/session tokens and private vault values never enter planner context.
17. RawScene never becomes a planner transport payload.
18. SafeContext is the normal local-to-remote application contract.
19. ActionProposal is untrusted advice, never direct browser authority.
20. No arbitrary JavaScript, shell, unrestricted selector or generic executable planner action.
21. Page/web/OCR content is untrusted data, not policy instruction.
22. Re-ground immediately before important actions.
23. Private token resolution happens only after local authorization.
24. High-risk actions are locally gated and confirmation-gated where policy requires.
25. Important actions are verified against observed page-state change.
26. Privacy claims are proven on actual serialized bytes/log/storage surfaces using canaries.
27. Performance/security/privacy results are measured, not invented.
28. Architecture/schema/trust/dependency changes require human approval and ADR when material.
29. AI coding agents implement bounded tasks and may recommend, but may not silently redesign.
30. When uncertainty increases, authority decreases: re-observe, re-plan, ask user, or stop safely.
31. The SIH prototype proves the thesis; future company infrastructure must not derail the 20-day critical path.

N-Eye Canonical Control Loop



N-EYE AI needs context. Not your identity.